

“헬기 착륙장” 문제 풀이

부분문제 1, 2

크기가 정확히 k 인 헬기 착륙장의 개수는 2^k 이다. 각각의 동심원마다 어떤 색을 칠할 지 결정하는 방법이 두 가지이고, 그 선택이 서로 독립적이기 때문이다.

또한, 크기가 정확히 k 인 헬기 착륙장을 색칠하기 위해서는 총 $1 + 2 + \dots + k = \frac{k(k+1)}{2}$ 통의 페인트가 필요하다.

현재 $a + b$ 통의 페인트를 가지고 있으므로, $\frac{k(k+1)}{2} \leq a + b$ 이어야 한다. 이 부분문제에서는 $a + b \leq 100 + 100 = 200$ 이므로, $k \leq 19$ 임을 알 수 있다.

따라서 모든 크기가 19 이하인 헬기 착륙장들 (총 $2^1 + 2^2 + \dots + 2^{19}$ 개)을 순회하면서, 사용하는 빨강 페인트가 a 통 이하이고 파랑 페인트가 b 통 이하인 것의 개수를 세면 된다.

부분문제 3

앞선 부분문제에서 한 관찰에 의해, 헬기 착륙장의 크기 k 는 대략 $\sqrt{2(a+b)}$ 이하여야 함을 알 수 있다.

$\text{ways}[k][i][j]$ 를 크기가 정확히 k 인 헬기 착륙장 중 빨강 페인트를 정확히 i 통 사용하고, 파랑 페인트를 정확히 j 통 사용하는 헬기 착륙장의 수라고 정의하자.

기저사례는 $\text{ways}[1][1][0] = \text{ways}[1][0][1] = 1$, 그 외의 $\text{ways}[1][i][j] = 0$ 이다.

$k \geq 2$ 인 경우, 크기가 정확히 k 인 헬기 착륙장 중 빨강 페인트를 정확히 i 통 사용하고, 파랑 페인트를 정확히 j 통 사용하는 방법은, 반지름이 k 인 동심원이 무슨 색인지에 따라 두 가지 경우로 나눌 수 있다.

- 빨강색인 경우: 크기가 $k - 1$ 이고 빨강 페인트를 $i - k$ 통, 파랑 페인트를 j 통 사용한 헬기 착륙장에, 반지름이 k 인 빨강색 원을 추가한 것이다. 경우의 수는 $\text{ways}[k - 1][i - k][j]$ 이다.
- 파랑색인 경우: 크기가 $k - 1$ 이고 빨강 페인트를 i 통, 파랑 페인트를 $j - k$ 통 사용한 헬기 착륙장에, 반지름이 k 인 파랑색 원을 추가한 것이다. 경우의 수는 $\text{ways}[k - 1][i][j - k]$ 이다.

따라서 $\text{ways}[k][i][j] = \text{ways}[k - 1][i - k][j] + \text{ways}[k - 1][i][j - k]$ 의 점화식이 성립한다.

원하는 것은 빨강 페인트를 a 통 이하, 파랑 페인트를 b 통 이하로 사용하는 모든 헬기 착륙장의 개수를 세는 것이므로, 정답은 $\sum_{k=1}^{90} \sum_{i=0}^a \sum_{j=0}^b \text{ways}[k][i][j]$ 이다.

모든 $k \leq \sqrt{2(a+b)} \approx 63$, $0 \leq i \leq a$, $0 \leq j \leq b$ 에 대해 ways 배열을 k, i, j 순서대로 계산할 수 있다. $k \cdot a \cdot b \leq 63 \cdot 1000 \cdot 1000 = 0.63 \times 10^8$ 이므로 제한 시간 내에 모두 계산 가능하며, 제한 메모리 안에 모든 값을 저장할 수 있다.

구현 시 아래 사항들을 주의해야 한다. 이후 부분문제들의 풀이는 아래 두 가지 주의사항을 염두에 두고 있다는 가정 하에 진행한다.

- i 또는 j 가 k 미만인 경우, 음수 인덱스를 참조하지 않도록 구현해야 한다.
- 헬기 착륙장의 수가 64비트 정수형의 범위를 넘을 수 있기 때문에, 덧셈을 할 때마다 $10^9 + 7$ 로 나눈 나머지를 취해야 한다.

부분문제 4

크기가 정확히 k 인 헬기 착륙장 중 빨강 페인트를 정확히 i 통 사용하고, 파랑 페인트를 정확히 j 통 사용하는 헬기 착륙장이 존재하려면, $i + j = \frac{k(k+1)}{2}$ 이어야 한다. 즉, i 와 j 가 결정되면 조건을 만족하는 k 의 값이 없거나, 유일하게 결정된다는 것을 알 수 있다.

이 점에서 착안해, $\text{ways}[i][j]$ 를 빨강 페인트를 정확히 i 통 사용하고, 파랑 페인트를 정확히 j 통 사용하는 헬기 착륙장의 수라고 정의하자. 앞선 부분문제와는 달리 k 가 상태 정의에서 제거되었다.

$i + j = \frac{k(k+1)}{2}$ 인 k 가 존재하지 않는다면, $\text{ways}[i][j] = 0$ 이다.

$i + j = \frac{k(k+1)}{2}$ 인 k 가 존재한다면, $\text{ways}[i][j] = \text{ways}[i][j - k] + \text{ways}[i - k][j]$ 이다.

원하는 것은 빨강 페인트를 a 통 이하, 파랑 페인트를 b 통 이하로 사용하는 모든 헬기 착륙장의 개수를 세는 것이므로, 정답은 $\sum_{i=0}^a \sum_{j=0}^b \text{ways}[i][j]$ 이다.

부분문제 5

크기가 정확히 k 인 헬기 착륙장 중 빨강 페인트를 정확히 i 통 사용하고, 파랑 페인트를 정확히 j 통 사용하는 헬기 착륙장이 존재하려면, $i + j = \frac{k(k+1)}{2}$ 이어야 한다. 즉 k 와 i 가 결정되면 $j = \frac{k(k+1)}{2} - i$ 이어야 한다.

$\text{ways}[k][i]$ 를 크기가 정확히 k 인 헬기 착륙장 중 빨강 페인트를 정확히 i 통 사용하고, 파랑 페인트를 정확히 $\frac{k(k+1)}{2} - i$ 통 사용하는 헬기 착륙장의 수라고 정의하자.

반지름이 k 인 동심원이 무슨 색인지에 따라 두 가지 경우로 나눌 수 있다.

- 빨강색인 경우: 크기가 $k - 1$ 이고 빨강 페인트를 $i - k$ 통 사용한 헬기 착륙장에, 반지름이 k 인 빨강색 원을 추가한 것이다. 경우의 수는 $\text{ways}[k - 1][i - k]$ 이다.
- 파랑색인 경우: 크기가 $k - 1$ 이고 빨강 페인트를 i 통 사용한 헬기 착륙장에, 반지름이 k 인 파랑색 원을 추가한 것이다. 경우의 수는 $\text{ways}[k - 1][i]$ 이다.

따라서 $\text{ways}[k][i] = \text{ways}[k - 1][i - k] + \text{ways}[k - 1][i]$ 의 점화식이 성립한다.

원하는 것은 빨강 페인트를 a 통 이하, 파랑 페인트를 b 통 이하로 사용하는 모든 헬기 착륙장의 개수를 세는 것이다. $\text{ways}[k][i]$ 가 정답에 포함되려면, $i \leq a$ 와 $\frac{k(k+1)}{2} - i \leq b$ 가 동시에 성립하면 되므로, 이러한 (k, i) 쌍들을 모두 순회하면서 그 합을 구하면 된다.

$k \leq \sqrt{2(a + b)}$ 이므로, 총 시간복잡도는 $O(a\sqrt{a + b})$ 이다.

부분문제 6

부분문제 5에서와 같이 $\text{ways}[k][i]$ 배열을 모두 구해 놓았다고 가정한다. 여기까지 총 $O(MAXA\sqrt{MAXA + MAXB})$ 시간이 필요하다.

$\text{ways}[k][i]$ 가 정답에 포함되려면, $i \leq a$ 이고 $\frac{k(k+1)}{2} - i \leq b$ 가 성립해야 한다. 두 부등식을 i 에 대해서 정리하면 $\max\left(0, \frac{k(k+1)}{2} - b\right) \leq i \leq a$ 와 같이 표현된다.

즉, 헬기 착륙장의 크기를 k 로 고정하면, 이 중 정답에 포함되는 경우의 수는 $\sum_{i=\max\left(0, \frac{k(k+1)}{2} - b\right)}^a \text{ways}[k][i]$ 와 같이 $\text{ways}[k]$ 배열의 구간 합으로 표현할 수 있음을 알 수 있다.

$\text{ways}[k]$ 배열의 부분합 배열 $\text{pref}[k][i] = \sum_{z=0}^i \text{ways}[k][z]$ 를 구해 두면, 위의 값을 $\text{pref}[k][a] - \text{pref}[k]\left[\max\left(0, \frac{k(k+1)}{2} - b\right) - 1\right]$ 로 나타낼 수 있으므로, 상수 시간에 구할 수 있다.

각각의 k 에 대해 상수 시간에 경우의 수를 구할 수 있으므로, 테스트 케이스 하나당 $O(\sqrt{a+b})$ 시간에 모든 경우의 수를 구할 수 있다.

“그래프 균형 맞추기” 문제 풀이

작성자: 김준원

부분문제 1

아주 간단한 형태의 그래프이므로 직접 예제 그림을 그려 생각할 수 있다. 간선의 가중치 조건이 입력으로 주어졌을 때, 정점의 가중치로 가능한 수는 유일하게 정해진다. 일반적으로

- 1번 정점과 2번 정점을 잇는 간선의 가중치가 c_{1-2}
- 2번 정점과 3번 정점을 잇는 간선의 가중치가 c_{2-3}
- 3번 정점과 1번 정점을 잇는 간선의 가중치가 c_{3-1}

로 주어지면, 정점에 부여할 가중치는 1번 정점에 $(c_{1-2} + c_{3-1} - c_{2-3})/2$, 2번 정점에 $(c_{2-3} + c_{1-2} - c_{3-1})/2$, 3번 정점에 $(c_{3-1} + c_{2-3} - c_{1-2})/2$ 으로 유일하게 정해진다. 이 값들을 직접 계산해서 출력하면 된다. 가중치는 정수여야만 하므로, 이 값들이 정수가 아니라면, No를 출력해야 함에 주의하라.

부분문제 2, 3, 4

부분문제 2, 3에서 입력 그래프는 직선이다. 여기서 i 번 정점과 $i+1$ 번 정점을 잇고 있는 간선의 가중치를 c_i 라고 하자.

- 만약 1번 정점에 가중치 x 를 부여했다면,
- 2번 정점에 가중치 $c_1 - x$ 를 부여해야 하고,
- 3번 정점에 가중치 $c_2 - c_1 + x$ 를 부여해야 하고,
- 4번 정점에 가중치 $c_3 - c_2 + c_1 - x$ 를 부여해야 하고,
- ...

와 같이, 한 정점에 부여할 가중치가 정해지면, 다른 각 정점에 부여할 가중치가 정해진다. 이 성질은 부분문제 4의 제약 조건을 만족하는 그래프(이런 그래프를 ‘트리’라고 부른다)에서도 마찬가지라서, 1번 정점의 가중치가 x 라고 할 때, 다른 각 정점 i 의 가중치는 $x + k_i$ 또는 $-(x + k_i)$ (k_i 는 상수)의 꼴로 정해진다. 이들 k_i 들은 부분문제 2, 3에서는 반복문을, 부분문제 4에서는 깊이 우선 탐색(DFS)을 이용해 구할 수 있다.

부분문제 1에서는 정점의 가중치가 유일하게 정해졌지만, 부분문제 2, 3, 4에서는 정점의 가중치를 부여하는 방법이 유일하지 않다. 1번 정점의 가중치 x 를 바꾸면, 다른 모든 정점의 가중치를 바꿀 수 있다. 반대로, 문제의 조건을 만족하도록 정점에 가중치를 부여하면, 이 가중치들은 모두 x 에 대한 식으로 쓸 수 있다.

이제 우리는 부분문제 2, 3, 4에서 정점에 가중치를 부여하는 방법은 무조건 존재한다(게다가, 무한정 많이 존재한다)는 사실을 안다. 이들 가중치를 모두 x 에 대한 식으로 쓸 수 있으므로, 가중치를 부여하는 비용 또한 아래와 같이 x 에 대한 식 $f(x)$ 로 쓸 수 있다.

$$f(x) = |x + k_1| + |x + k_2| + \cdots + |x + k_N|$$

함수 $f(x)$ 는 아래로 볼록한 꺾은선 모양으로 나타남을 관찰할 수 있다. 이 사실에서 x 가 N 개의 수 $-k_1, -k_2, \dots, -k_N$ 중 하나일 때 최솟값을 가진다는 사실을 알 수 있다. 이들 후보들을 모두 직접 시험해서 가장 작은 비용을 출력하면 $O(N^2)$ 에 부분문제 2를 풀 수 있다.

그리고 추가적인 관찰을 통해 x 가 N 개의 수 $-k_1, -k_2, \dots, -k_N$ 의 중간값일 때에 최솟값을 가진다는 사실을 알 수 있다. 이를 이용해 최적의 x 를 바로 알 수 있고, $O(N \log N)$ 에 부분문제 3, 4를 풀 수 있다.

부분문제 5

부분문제 5에서 입력 그래프는 목걸이 모양이다(즉, 하나의 ‘사이클’로 이루어져 있다). 즉 그래프는, 아래 그래프에서 정점과 간선의 순서를 임의로 섞은 꼴이다.

- i ($1 \leq i \leq N-1$)번째 간선은 i 번 정점과 $i+1$ 번 정점을 잇고 있으며, 가중치는 c_i 이다.
- N 번째 간선은 N 번 정점과 1번 정점을 잇고 있으며, 가중치는 c_N 이다.

정점의 순서를 섞어도 본질은 변하지 않기 때문에, 위와 같은 그래프가 입력으로 주어졌다고 하자.

부분문제 2, 3, 4의 아이디어와 같이 1번 정점에 x 의 가중치를 부여하고 나머지 정점에 부여할 가중치를 x 에 대한 식으로 나타낸다. 마지막 N 번째 간선을 무시하면 부분문제 2, 3에서 등장했던 직선형 그래프가 된다. 따라서 각 정점의 가중치를 x 에 대한 식으로 나타낼 수 있다. 하지만 N 번째 간선의 존재 때문에 확인할 게 하나 더 생긴다. 그래프의 크기(사이클의 크기)의 홀짝성에 따라 두 가지 경우로 나눌 수 있다.

사이클의 크기가 짝수일 때, N 번 정점은 $-(x + k_N)$ 꼴의 가중치를 가지게 된다. N 번째 간선에 의해 $x + -(x + k_N) = c_N$ 이라는 조건이 추가된다.

- 만약 $k_N \neq c_N$ 이라면 그래프의 균형이 맞도록 가중치를 부여할 수 있는 방법이 없고,
- 그렇지 않다면 N 번째 간선은 무시할 수 있다.

사이클의 크기가 홀수일 때, N 번 정점은 $x + k_N$ 꼴의 가중치를 가지게 된다. N 번째 간선에 의해 $x + (x + k_N) = c_N$ 이라는 조건이 추가된다. 이 조건에 의해 $x = (c_N - k_N)/2$ 가 되어야만 한다.

- 만약 x 가 정수가 아니라면, 그래프의 균형이 맞도록 정수 가중치를 부여할 수 있는 방법이 없고,
- x 가 정수라면, $x = (c_N - k_N)/2$ 를 각 식에 대입했을 때의 가중치가 유일한 답이 된다.

부분문제 6, 7

부분문제 2, 3, 4와 부분문제 5를 풀었다면, 전체 문제를 푼 것이나 다름없다. 풀이의 줄거리는 다음과 같다.

1번 정점에 가중치 x 를 부여했을 때, 다른 모든 정점 i 에 부여할 가중치가 간선을 따라 $x + k_i$ 또는 $-(x + k_i)$ 의 꼴로 정해진다. 하지만 그래프에 사이클이 있다면, 입력 조건들 사이 ‘충돌’이 발생할 수 있다. 충돌은 부분문제 5와 같이 경우를 나누어 해결해줄 수 있다. 충돌을 처리한 결과, i) 불가능함이 증명되거나, ii) x 의 값이 유일하게 정해지거나, iii) x 의 값이 정해지지 않을 수 있다. i), ii)의 경우에는 해당하는 답을 바로 출력할 수 있고, iii)의 경우에는 부분문제 2, 3, 4의 풀이와 같이 x 를 $-k_1, \dots, -k_N$ 의 중간값으로 두어 비용을 최소화할 수 있다.

“최대 길이 공통 괄호 문자열” 문제 풀이

작성자: 최은수

부분문제 1

S 의 부분 문자열인 괄호열을 S 의 부분 괄호열이라 하자. 모든 괄호열은 여는 괄호 하나에서 시작해 닫는 괄호 하나에서 끝나기 때문에 A 의 부분 괄호열의 개수는 많아야 $\frac{|A|^2}{4}$ 개이다. 마찬가지로, B 의 부분 괄호열의 개수는 많아야 $\frac{|B|^2}{4}$ 개이다.

A 의 부분 괄호열 중, B 의 부분 괄호열인 것만을 모아, 이 중 최대 길이인 것을 구하면 $O(|A|^5 + |B|^5)$ 에 문제를 해결할 수 있다.

부분문제 2

A 의 부분 괄호열과 B 의 부분 괄호열 쌍 모두를 비교하는 것이 아니라, 각각을 정렬하여 비교하면 $O(|A|^3 + |B|^3)$ 에 문제를 해결할 수 있다.

부분문제 3

A 의 모든 접미사에 대한 트라이를 만들고, 이 중 올바른 괄호열인 문자열에 해당하는 노드에 표시를 하자.

B 의 모든 접미사를 트라이에서 검색하여, 검색하며 만나는 표시된 노드 중 깊이가 가장 큰 노드의 깊이가 답이 된다.

트라이의 구성 및 검색에는 문자열의 길이의 합만큼의 시간이 소요되므로, 시간복잡도는 $O(|A|^2 + |B|^2)$ 이다.

부분문제 4

어떤 문자열 S 가 있을 때, S 의 i 번째 문자까지 있는 여는 괄호의 개수에서 닫는 괄호의 개수를 뺀 값을 문자열 S 의 i 번째 높이 h_i 로 정의하자. 편의상 $h_0 = 0$ 으로 정의하자.

$A[i..j]$ 가 올바른 괄호열인 조건과, $h_{i-1} = h_j$ 이며, $i \leq k < j$ 를 만족하는 모든 k 에 대해 $h_k \geq h_j$ 인 조건은 동치이다.

$1 \leq i \leq |A|$ 를 만족하는 i 에 대해, 다음 방법을 통해 $A[i..j]$ 가 올바른 괄호열인 가장 큰 j 를 구할 수 있다.

먼저, 이분탐색을 통해 $i \leq k$ 이면서, $h_{i-1} - 1 = h_k$ 인 가장 작은 k 를 찾는다. 없다면 $k = |A|$ 로 둔다.

다음으로, $j \leq k$ 이면서, $h_{i-1} - 1 = h_k$ 인 가장 큰 j 를 찾는다. 만약 $j = i - 1$ 이라면 i 번째 문자에서 시작하는 올바른 괄호열은 없다.

이런 $A[i..j]$ 들 중 가장 긴 문자열의 길이가 답이 된다. 시간복잡도는 $O(|A| \log |A|)$ 이다.

부분문제 5

두 문자열 A 와 B 를 합친 $A * B$ 의 접미사 배열을 구하자. 그 후, 이 접미사 배열의 LCP 배열을 구하자.

A 의 접미사 $A[i..|A|]$ 에 대해, 이 문자열의 B 의 부분 문자열인 가장 긴 접두사는 다음의 두 가지 경우이다.

- 사전순으로 $A[i..|A|]$ 보다 앞인 B 의 접미사 중 가장 뒤에 있는 접미사와의 최장 공통 접두사

- 사전순으로 $A[i..|A|]$ 보다 뒤인 B 의 접미사 중 가장 앞에 있는 접미사와의 최장 공통 접두사

두 경우의 길이 모두 스위핑을 통하여 모든 접미사에 대해 $O(|A| + |B|)$ 에 구할 수 있으며, 둘 중 최댓값의 배열을 $l_1, l_2, \dots, l_{|A|}$ 라 하자.

답의 경우 부분문제 4의 마지막 이분탐색에서 $j < i + l_i$ 라는 조건만 추가되었기 때문에, k 대신 $\min(k, i + l_i - 1)$ 를 사용하여 구할 수 있다. 시간복잡도는 $O(|A| \log |A| + |B| \log |B|)$ 이다.

“맛집 추천” 문제 풀이

작성자: 윤창기

부분문제 1

부분문제 1은 트리가 직선인 특수한 경우이다. 이 경우, R_i 들을 모두 구간으로 생각할 수 있다.

$1 \leq i \leq N$ 에 대해 D_i 를 “선택한 모든 구간들이 i 왼쪽으로 넘어가지 않으면서 서로소일 때 (서로 겹치지 않을 때) 가능한 최대 만족도”라고 정의하자. 문제의 답은 D_1 이 됨을 쉽게 알 수 있다. 모든 D_i 의 값을 동적 계획법으로 구해보자.

언급한 조건에서 i 번 정점은 구간으로 덮지 않거나, 어떤 $[i, j]$ 꼴의 구간을 사용하여 덮어야 한다.

- i 번 정점을 구간으로 덮지 않는 경우 얻을 수 있는 최대 만족도는 D_{i+1} 이다.
- 어떤 구간 $[i, j]$ 를 사용했을 때 g 의 만족도를 얻을 수 있다면, 이 구간을 사용했을 때 가능한 최대의 만족도는 $D_{j+1} + g$ 이다.

이 경우를 모두 고려하면 $O(M + N)$ 시간에 문제를 해결할 수 있다.

부분문제 2

부분문제 2는 맛집의 개수 M 이 20 이하로 매우 작은 경우이다. 따라서 가능한 모든 맛집의 집합을 탐색하면서 문제의 조건을 만족하는지 판별하는 방법으로 문제를 해결하기에 충분하다.

선택된 맛집들에 대해, 너비 우선 탐색 등을 이용하여 거리가 d_i 인 점들을 모두 구할 수 있다. 이 중 겹치는 점이 있는지 판단하는 방법으로 $O((N + M)2^M)$ 시간에 문제를 해결할 수 있다.

혹은, 두 맛집 $i \neq j$ 가 동시에 선택되지 못할 조건을 $d(i, j) \leq d_i + d_j$ 로 나타낼 수 있으므로, 모든 맛집에 대해 $d(i, j)$ 를 $O(MN)$ 또는 $O((M^2 + N) \log N)$ 시간에 구하는 방법으로 $O(N \log N + M^2 2^M)$ 시간에 문제를 해결할 수도 있다.

부분문제 3

부분문제 3은 $M, N \leq 2000$ 인 경우이다. 따라서 $O(MN)$ 시간에 문제를 해결하면 충분하다.

임의로 1번 정점을 루트로 잡고, 부분문제 1의 관찰을 일반화하여 D_v 를 “ v 를 루트로 하는 서브트리에서, 선택한 모든 R_i 들이 v 의 서브트리에 포함되며 서로소일 때 가능한 최대 만족도”로 정의해보자. 문제의 답이 D_1 임을 확인할 수 있다. 마찬가지로 경우를 분할하여 생각해보면

- v 가 어떤 R_i 들로도 덮이지 않는 경우, 가능한 최대 만족도는 v 의 모든 자식 u 에 대해 D_u 의 합이다.
- v 가 R_i 로 덮이는 경우, R_i 는 v 를 포함하면서 v 의 서브트리 밖을 나가면 안 된다. v 의 서브트리에서 R_i 를 지우면, 이 서브트리가 여러 개로 분할되며, 각 서브트리 조각의 루트는 $d(c_i, x) = d_i + 1$ 을 만족하는 x 이다. 가능한 최대 만족도는 v 의 서브트리에 속한 점들 중 $d(c_i, x) = d_i + 1$ 을 만족하는 x 에 대해 D_x 를 합한 값이다.

이 때 v 가 R_i 로 덮이면서, v 의 서브트리 밖이 R_i 로 덮이지 않으려면, v 는 R_i 가 덮는 점들 중 가장 루트에 가까워야 한다. 따라서 각 R_i 를 고려하는 시점은 v 가 c_i 의 d_i 번째 조상일 때로 유일하며, 이는 DFS나 자료구조를 활용하여 빠른 시간 안에 구할 수 있다.

$d(c_i, x) = d_i + 1$ 을 만족하는 정점의 개수가 $O(N)$ 개이므로, 전체 시간복잡도 $O(MN)$ 에 문제를 해결할 수 있다.

부분문제 4

부분문제 4는 $N \leq 2000$ 인 경우이다. $(c_i, d_i) = (c_j, d_j)$ 인 두 맞집 i, j 가 있다면, 둘 중 만족도가 큰 것만 고려해도 관계가 없다.

$\mathcal{R}(c, t)$ 를 $d(c, x) = t$ 를 만족하는 정점 x 의 개수라고 하자. 부분문제 3의 동적 계획법을 채우기 위해 필요한 시간복잡도는 $O(N + M + \sum_i \mathcal{R}(c_i, d_i + 1))$ 임을 알 수 있다.

이 때, $\sum_{d=1}^{N-1} \mathcal{R}(c, d) = N - 1$ 이고 $(c_i, d_i + 1)$ 이 모두 다르므로,

$$\sum_i \mathcal{R}(c_i, d_i + 1) \leq \sum_{c=1}^N \sum_{d=1}^{N-1} \mathcal{R}(c, d) = \sum_{c=1}^N (N - 1) = N(N - 1)$$

의 부등식이 성립한다. 따라서 전체 시간 $O(N^2 + M)$ 에 문제를 해결할 수 있다.

부분문제 5

부분문제 5는 도시가 완전 이진 트리 모양인 경우이다. 역시 부분문제 3의 동적 계획법을 개선하여 문제를 해결할 수 있다.

어떤 정점 z 의 부모를 $p(z)$ 로 쓰자. $d(c, x) = d + 1$ 을 만족하는 점 x 들은 다음의 겹치지 않는 조건들 중 하나를 만족한다. v 는 $R(c, d)$ 가 덮는 가장 루트에 가까운 점이므로, $d(c, v) = d$ 임에 주목하자.

- c 의 서브트리에 속하고, 깊이가 $(\text{dep}_c: c\text{의 깊이}) + d + 1$ 이다.
- $p(c)$ 의 (c 와는 다른) 서브트리에 속하고, 깊이가 $\text{dep}_{p(c)} + d$ 이다.
- $\vdots$
- $p^j(c)$ 의 ($p^{j-1}(c)$ 와는 다른) 서브트리에 속하고, 깊이가 $\text{dep}_{p^j(c)} + d + 1 - j$ 이다.
- $\vdots$
- $v = p^d(c)$ 의 ($p^{d-1}(c)$ 와는 다른) 서브트리에 속하고, 깊이가 $\text{dep}_v + 1$ 이다.
- $p(v)$ 이다. D_v 를 구하는 데는 기여하지 않으므로 무시해도 좋다.

따라서, $F(v, d)$ 를 v 의 서브트리 중 v 와의 거리가 d 인 정점 x 에 대한 D_x 의 합으로 정의하면, 위의 각 경우에 해당하는 D_x 의 합을 $F(v, d)$ 들을 이용해 나타낼 수 있다.

- c 의 서브트리에 속하고, 깊이가 $\text{dep}_c + d + 1$ 이다.
 $\rightarrow F(c, d + 1)$
- $p^j(c)$ 의 ($p^{j-1}(c)$ 와는 다른) 서브트리에 속하고, 깊이가 $\text{dep}_c + d + 1 - j$ 이다. ($1 \leq j \leq k$)
 $\rightarrow F(p^j(c), d + 1 - j) - F(p^{j-1}(c), d + 1 - j)$

따라서 $F(v, d)$ 를 빠른 시간 안에 관리할 수 있으면 문제를 해결할 수 있다. 이 부분문제의 경우 $d \leq \log N$ 이므로, 매번 $O(\log N)$ 의 $F(\cdot, \cdot)$ 값만 사용하거나 수정하여 동적계획법을 수행할 수 있다. 따라서 시간복잡도 $O(M \log N)$ 에 문제를 해결할 수 있다.

부분문제 6

부분문제 6은 트리에 차수 3 이상인 정점이 하나뿐인 경우이다. 일반성을 잃지 않고 그 정점을 1번 점이라 두자. 1번 점을 루트로 하는 트리를 생각했을 때, 트리의 특성에 의해 루트에 연결된 모든 서브트리가 직선 모양이 되는 것을 알 수 있다. 직선 e 에 속한 정점들을 루트에서부터 가까운 순서로 $e(1), e(2), \dots$ 라고 쓰자.

따라서 부분문제 1의 간단한 동적 계획법을 각 직선에 적용하여 $F(1, d) = \sum_e D_{e(d)}$ 를 선형 시간에 구할 수 있다. D_1 을 구하기 위해, 1번 정점을 R_i 로 덮는 상황을 생각해 보자. $b = d(c_i, 1)$ 이라고 두면, R_i 는 c_i 가 속한 직선 e 에서는 $e(d_i + b)$ 까지, 다른 직선 e' 에서는 $e'(d_i - b)$ 까지를 덮는다. 따라서 얻을 수 있는 만족도의 최댓값은 아래와 같은 식으로 나타난다.

$$\sum_{e' \neq e} D_{e'(d_i - b + 1)} + D_{e(d_i + b + 1)} + g_i = F(1, d_i - b + 1) - D_{e(d_i - b + 1)} + D_{e(d_i + b + 1)} + g_i$$

따라서, 시간복잡도 $O(M + N)$ 에 문제를 해결할 수 있다.

부분문제 7

부분문제 5의 풀이는 일반적인 경우 d 가 $N - 1$ 까지 커질 수 있으므로 적용하기에 어려움이 있다. 이를 최적화하기 위해 트리의 센트로이드 분할 (Centroid decomposition)을 생각하자.

정점 c 가 트리 T 의 센트로이드(centroid)라는 것은, c 를 루트로 했을 때 가장 큰 서브트리의 크기가 전체 정점의 절반을 넘지 않는다는 것을 말한다. 한 트리에서 센트로이드는 1개 또는 2개 존재하며, 선형 시간에 찾을 수 있다. 반복적으로 서브트리의 센트로이드를 찾아서 트리를 분할하는 과정을 센트로이드 분할이라고 하며, 센트로이드의 정의상 총 $O(N \log N)$ 시간에 수행할 수 있다.

편의상 센트로이드 분할 과정을 트리 형태의 도식으로 나타낼 수도 있다. 즉, 전체 트리에서 센트로이드 c 를 기준으로 트리를 분할하고, 한 서브트리의 센트로이드 d 를 기준으로 서브트리를 분할하고, 다시 e 를 ... 와 같은 방식으로 센트로이드 분할이 이루어졌다면 c 가 d 의 부모, d 가 e 의 부모, ...가 되도록 트리를 구성하는 방식이다. 이렇게 만든 도식을 ‘센트로이드 트리’ T^* 라고 부르자. T^* 는 항상 깊이가 $O(\log N)$ 인 좋은 성질을 갖는다. 이후, 부분문제 5의 관찰과 유사한 흐름으로 전체 문제를 해결할 수 있다.

T^* 에서 어떤 정점 z 의 부모를 $\pi(z)$ 로 쓰자. $d(c, x) = d + 1$ 을 만족하는 점 x 는 다음의 겹치지 않는 조건들 중 하나를 만족한다.

- c 의 서브트리에 속하고, $d(c, x) = d + 1$ 이다.
- T^* 에서 $\pi(c)$ 의 (c 와는 다른) 서브트리에 속하고, $d(\pi(c), x) + d(\pi(c), c) = d + 1$ 이다.
- $\vdots$
- $\pi^j(c)$ 의 ($\pi^{j-1}(c)$ 와는 다른) 서브트리에 속하고, $d(\pi^j(c), x) + d(\pi^j(c), c) = d + 1$ 이다.
- $\vdots$
- T^* 의 루트 (센트로이드) $r = \pi^k(c)$ 의 ($\pi^{k-1}(c)$ 와는 다른) 서브트리에 속하고, $d(r, x) + d(r, c) = d + 1$ 이다.

항상 $k \leq \log N$ 임에 주목하자. 이번에도 $F(v, d)$ 와 비슷한 값을 정의하여 각 경우를 닫힌 꼴로 나타낼 것이다. $F^*(v, d)$ 를 T^* 에서 v 의 서브트리 중 v 와의 거리가 d 인 정점 x 에 대한 D_x 의 합, $G^*(v, d)$ 를 T^* 에서 $\pi(v)$ 의 v 가 속한 서브트리 중 $\pi(v)$ 와의 거리가 d 인 정점 x 에 대한 D_x 의 합이라고 두면,

- c 의 서브트리에 속하고, $d(c, x) = d + 1$ 이다.

$$\rightarrow F^*(c, d + 1)$$

- $\pi^j(c)$ 의 ($\pi^{j-1}(c)$ 와는 다른) 서브트리에 속하고, $d(\pi^j(c), x) + d(\pi^j(c), c) = d + 1$ 이다. ($1 \leq j \leq k$)

$$\rightarrow F^*(\pi^j(c), d + 1 - d(\pi^j(c), c)) - G^*(\pi^{j-1}(c), d + 1 - d(\pi^j(c), c))$$

따라서 $F^*(v, d)$, $G^*(v, d)$ 의 값만 빠른 시간 안에 관리할 수 있다면 문제를 해결할 수 있다. 이 때, 센트로이드 분할 과정을 생각해보면 $F^*(v, d) > 0$ 혹은 $G^*(v, d) > 0$ 인 (v, d) 의 개수가 많아야 $O(N \log N)$ 개이다. 따라서 $F^*(v, \cdot)$, $G^*(v, \cdot)$ 를 배열로 관리하면 전체 문제를 $O((N + M) \log N)$ 에 해결할 수 있다.